

Odoo Internals — Reference

A keepable reference on how Odoo works under the hood — the core mental model, the file-to-HTML rendering pipeline, and how to predict where any string points. Authored alongside the QMS learning module (Odoo 19).

Odoo Core — The Remember Map

One page to "grab all of Odoo." Everything below hangs off **ONE idea**. Learn the idea, and every feature becomes an obvious consequence of it.

★ THE ONE IDEA (memorize this first)

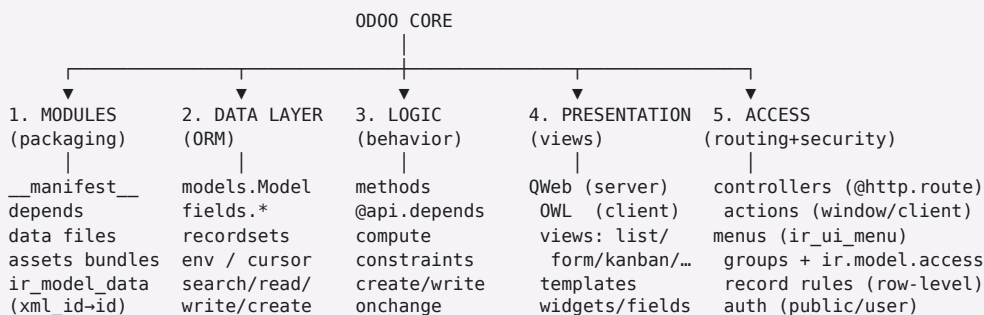
EVERYTHING IS A RECORD,
and ONE GENERIC ENGINE INTERPRETS RECORDS into a running app.

You don't build an app. You DECLARE records (in files).
Odoo stores them, and a generic ORM + generic web client RUN them.

From this single idea, everything else is a consequence:

- A **model** is a record (`ir_model`). A **view** is a record (`ir_ui_view`). A **menu**, an **action**, a **field**, even a **translation** — all rows in the database.
- Because the *app itself* is data, **one** web client can render **any** module without custom code.
- The developer's job = **write declarative data** + **a little Python logic**. Odoo does the rest.

🌳 THE CORE, AS A TREE (the mind-map)



Below each branch is "what Odoo takes care of for you" — that's the whole framework.

🔌 THE SPINE — the one through-line that connects everything

FILES —install→ REGISTRY (RAM) + DATABASE (rows) —ORM→ RPC (JSON) → OWL → HTML
.py/.xml code+metadata data+UI-config serialize string compile DOM

Source of truth = REGISTRY (Python) + DATABASE (Postgres). Everything else is a projection.

If you remember only one line, remember the spine. Every question ("how does X work?") is just "where on the spine does X live?"

🧠 "HOW DOES ODOO TAKE CARE OF ___ ?" (the cheat-sheet)

Concern	Odoo's mechanism	Lives where
Define data	<code>models.Model</code> + <code>fields.*</code> in a <code>.py</code>	registry (RAM) + DB columns
Store data	the ORM auto-creates/updates tables	PostgreSQL
Query data	<code>env[model].search(domain)</code> / read / browse	server (Python)
Save data	<code>create()</code> / <code>write()</code> (override for logic)	server → DB
Derived values	<code>compute=</code> + <code>@api.depends</code> (stored or not)	server (run on read)
Rules/validation	<code>models.Constraint</code> (SQL) + <code>@api.constrains</code> (Python)	server / DB
Reactive form behavior	<code>@api.onchange</code>	server, called per edit
Build the UI	declarative views (<code><list>/<form>/<search></code>) = records	DB (<code>ir_ui_view</code>)
Render backend UI	OWL compiles arch → DOM (client-side)	browser
Render public pages/reports	QWeb renders HTML (server-side)	server
Navigate	menus → actions (<code>act_window</code> / client action)	DB records
Talk to server (data)	JSON-RPC <code>call_kw</code> (orm service)	network
Real-time push	WebSocket bus (<code>bus.bus._sendone</code>)	network (:8072)
Public web routes	<code>@http.route(type="http")</code> controllers	server
Who can see/do what	groups + <code>ir.model.access</code> (CRUD) + record rules (rows)	DB, enforced by ORM
Multi-company / multi-user	the environment (<code>env</code>) carries user + context	server
Translations	<code>_t()</code> (JS) / <code>_()</code> (Python) + <code>.po</code> files	both
Reports (PDF)	QWeb report templates → wkhtmltopdf	server
Wire it together	shared names + XML-ids (<code>ir_model_data</code>)	DB
Package & ship	an addon with <code>__manifest__.py</code>	files → install

Read this table top-to-bottom once and you have seen *all* of Odoo's core responsibilities.

🗄️ THE META-TABLES (Odoo describing itself)

Odoo stores its **own structure** as data. These `ir_*` tables ARE the framework:

```
ir_model           → "what models exist"           (qms.queue is a row here)
ir_model_fields    → "what fields exist"          (q_name, quid... rows)
ir_ui_view         → "the views" (arch_db = your XML as TEXT)
ir_ui_menu         → "the menus"
ir_actions_act_window / ir_actions_client → "the actions"
ir_model_data      → "xml_id → row id" (the GLUE that links files & records)
ir_model_access    → "group CRUD permissions"
ir_rule            → "row-level record rules"
ir_cron            → "scheduled jobs"
ir_attachment      → "files/blobs + generated asset bundles"
```

Insight: when you "create a model/view/menu," you are **inserting rows into these tables**. That is why a generic engine can run any app — it just reads these rows.

THE REQUEST LIFECYCLE (compact, the runtime loop)

1. Browser sends HTTP → werkzeug → ir_http._dispatch (routing by URL)
2. Auth check (public/user) + session
- 3a. type="http" → controller returns HTML/file/redirect (website pages, reports)
- 3b. type="jsonrpc" /web/dataset/call_kw → call_kw(model,method) (backend data; RPC)
 - └ get_public_method (no leading "_") → getattr(model, method)
 - └ ORM runs: search/read/write/compute + SECURITY (ACL + record rules)
4. Result serialized to JSON → back to browser
5. OWL updates the reactive model → re-renders the DOM
(parallel) WebSocket bus pushes live events when the server calls _sendone

DEVELOPER WRITES → WHAT IT BECOMES

<code>_name = "qms.queue"</code>	→ a model in registry + table qms_queue + ir_model row
<code>q_name = fields.Char()</code>	→ a column + ir_model_fields row + view metadata
<code>def create(self): ...</code>	→ business logic run on every insert
<code><list><field name="q_name"/></code>	→ ir_ui_view.arch_db (TEXT) → OWL component at runtime
<code><record id="action_qms_queue"></code>	→ ir_actions_act_window row + ir_model_data('...')
<code><menuitem action="..."></code>	→ ir_ui_menu row
<code>@http.route("/qms/book")</code>	→ a URL in the routing map → returns QWeb HTML
<code>"assets": {"web.assets_backend":...}</code>	→ your JS/CSS bundled into the page
<code>depends = ["base","website"]</code>	→ install order + available features

The whole craft of Odoo dev = choosing the right declarative record + a little Python.

THE TWO RENDERING ENGINES (don't confuse them)

SERVER-SIDE (SSR)	CLIENT-SIDE (CSR)
Engine : QWeb (Python)	Engine : OWL (JavaScript, @odoo/owl)
For : website pages, reports, email	For : the backend web client, all backend views
Output : HTML string	Output: live, reactive DOM
Trigger: request.render()	Trigger: mountComponent()
Same t-esc/t-if/t-foreach syntax – but QWeb runs ONCE on the server, OWL runs reactively in the browser (and has t-on-click events).	

THE THREE WAYS BROWSER ↔ SERVER TALK (chosen by NEED, not by UI)

NEED a full page	→ plain HTTP GET/POST	→ server-rendered HTML	(website)
NEED data on demand	→ JSON-RPC (call_kw)	→ request/response	(backend + website)
NEED a live push	→ WebSocket (bus)	→ server → browser	(Discuss, livechat)

NOT REST. Odoo is RPC ("call a method on a model"), not resource URLs.

THE MNEMONIC (carry this in your head)

"Declare records, ORM serves them, one client renders them."

Or the spine:

Files → Registry + DB → ORM → RPC → OWL → DOM.

Everything in Odoo is one of:

1. a **record** you declared (model/view/menu/action),

2. a bit of **Python logic** on a model,
3. the **generic engine** (ORM + web client) that interprets 1 and runs 2.

When confused, ask: *"Is this a record, logic, or the engine? And where on the spine does it live — registry, DB, network, or browser?"* That question answers almost everything.

Companion doc

See `odoo_rendering_pipeline.md` (same folder) for the **deep, file:line-grounded trace** of how one `<field>` becomes an HTML `<td>` — the detailed version of "the spine."

Odoo Rendering Pipeline — from `.xml`/`.py` files to native HTML

The journey nobody documents: how a developer's **XML view** + **Python model** become **records in a database**, get **fragmented** and **serialized**, travel over **RPC** as a **string**, and are turned by **OWL** into **native HTML DOM** in the browser.

Grounded in Odoo 19 source (file:line references are real).

0. The 30,000-ft timeline — TWO phases

PHASE A — INSTALL / STARTUP (happens ONCE, server-side, no browser)
Developer FILES $\xrightarrow{\text{digest}}$ REGISTRY (RAM) + DATABASE RECORDS
After this, the files are never read again at runtime.

PHASE B — RUNTIME (every page interaction)
Browser $\xrightarrow{\text{RPC}}$ Server reads REGISTRY+DB $\xrightarrow{\text{JSON string}}$ OWL $\rightarrow$ HTML

The two "pure" things produced in Phase A are the only source of truth:

- **Python registry** (model classes + fields + logic) — in RAM, rebuilt from code each boot.
- **PostgreSQL** (records: business data + UI config like the view arch) — on disk.

1. PHASE A — INSTALL: files become a registry + database records

DEVELOPER WRITES (files on server disk)

`__manifest__.py` $\leftarrow$ the INDEX: lists which files load + order
`qms_queue.py` $\leftarrow$ Python model (`q_name`, `quid`, logic)
`qms_queue_views.xml` $\leftarrow$ view arch (`<list>`, `<form>`, `<search>`)
`qms_menus.xml` $\leftarrow$ menus
(actions inside views)

$\rightarrow$ `odoo-bin -i qms` (install)

load Python $\rightarrow$ REGISTRY (in RAM)

`qms_queue.py`
`class QmsQueue`
`_name="qms.queue"`
`q_name=fields.Char()`

"qms.queue" $\rightarrow$ `QmsQueue` class
fields: `quid`, `q_name`, ...
methods: `create/write/...`

also CREATES SCHEMA in DB: `CREATE TABLE qms_queue (quid int, q_name varchar, ...)`

load XML data $\rightarrow$ DATABASE RECORDS (rows)

`qms_queue_views.xml`
`<record model="ir.ui.view">`
`<field name="arch">`
`<list>...</list>`

`ir_ui_view`
`id=512 model='qms.queue' type='list'`
`arch_db = "<list><field name=\"q_name\"`
`/>...</list>"` $\leftarrow$ XML as TEXT

`qms_menus.xml`
actions

`ir_ui_menu id=88 action=...`
`ir_actions_act_window res_model='qms.q...`

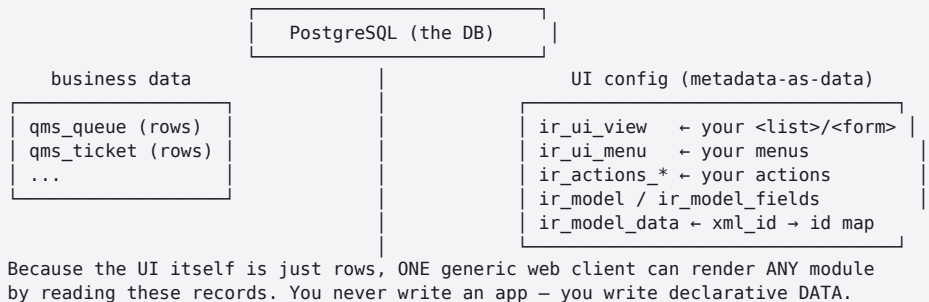
every `id="..."`

`ir_model_data` (the XML-id $\leftrightarrow$ row-id map)
`'qms.action_qms_queue'  $\rightarrow$  301`
`'qms.view_qms_queue_form'  $\rightarrow$  513`

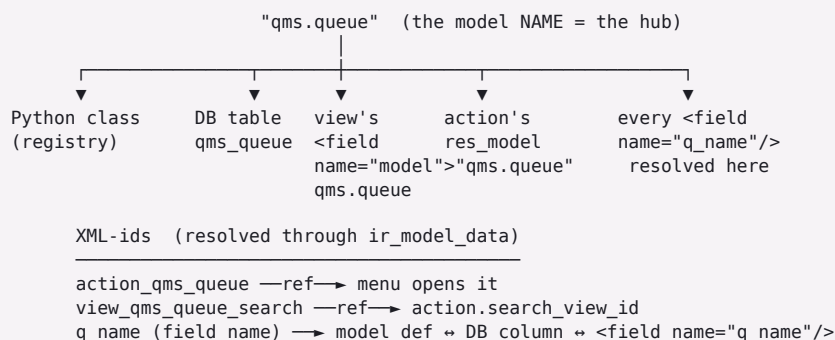
Key fragmenting facts:

- The view **arch XML becomes a TEXT string** in `ir_ui_view.arch_db` (`odoo/addons/base/models/ir_ui_view.py:162` `arch_db = fields.Text(...)`).
- The model's existence/fields become rows in `ir_model` / `ir_model_fields`.
- Every `id="..."` is recorded in `ir_model_data` — the table that resolves `module.xml_id → database row id`. **This is the glue table.**

2. The "EVERYTHING IS A RECORD" map (why one engine renders any app)



3. The GLUE — names & XML-ids connect the dots



The developer connects everything by **reusing names**. Odoo resolves those names via the **registry** (for model/field names) and `ir_model_data` (for XML-ids).

4. PHASE B — RUNTIME: the request→HTML pipeline (the main event)

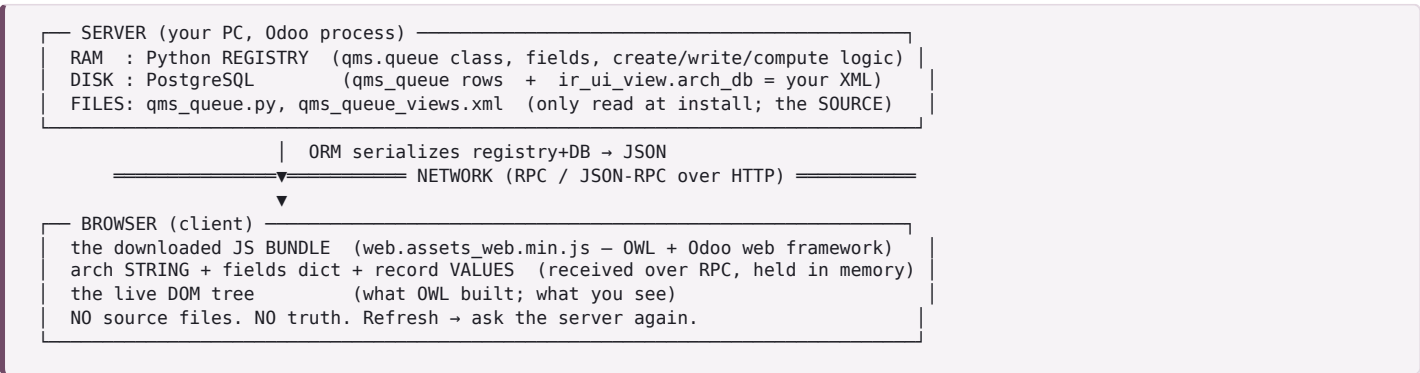


5. THE HEART — how ONE <field> transforms, stage by stage

STAGE	WHERE	WHAT IT IS
1. authored	dev file qms_queue_views.xml	<field name="q_name"/> (you type this)
install		
2. stored	server DB ir_ui_view	arch_db = "...<field name=\"q_name\"/>..." (TEXT in a row)
runtime: get_views() reads arch + fields_get()		
3. serialized	RPC (wire)	{ arch:"<field name='q_name'/>", fields:{ q_name:{type:'char',string:'Name',required:true} } } (a STRING + a dict)
4. compiled-1	browser JS view_compiler.js	<Field name="q_name" type="char"/> (OWL component tag) [Odoos translates arch→OWL]
5. compiled-2	browser JS owl.js compileTemplate	function render(ctx){ ... CharField ... } (a JS function) DOMParser + new Function [OWL]
6. rendered	browser RAM owl.js render()	VDOM: { tag:'td', text: ctx.record.data.q_name } (virtual)
+ data q_name = "Counter A"		
7. mounted	browser DOM document.createElement	<td>Counter A</td> (REAL HTML node)
8. displayed	screen	Counter A (pixels)

The browser is handed your XML **never**. It is handed a JS bundle that, at stage 5-7, **creates the HTML itself** with `document.createElement`. "Browser knows only HTML" holds — OWL is JS, and JS *makes* HTML.

6. WHERE EVERYTHING LIVES (memory map)



7. WHO DOES WHAT (division of labor)

BROWSER	: runs JS, knows HTML/CSS, executes document.createElement
OWL	: template STRING → JS function → VDOM → real DOM (the generic engine)
ODOO	: the bridge that feeds OWL — - server: get_views + fields_get (ship arch string + field metadata) - client: view_compiler (arch → OWL template), fields registry (type → widget), ListRenderer/FormRenderer, RelationalModel (reactive data), rpc/orm
DEVELOPER	: writes declarative .xml + .py using consistent NAMES

8. Mermaid — the runtime sequence (renders in VS Code / GitHub)

9. Mermaid — install-time fragmentation

10. One-sentence summary

The developer writes **files full of consistent names**; at **install** Odoo compiles them into a **Python registry** (code) + **database records** (the view arch stored as a *string*), linked by those names via `ir_model_data`; at **runtime** a **generic engine** serializes registry+DB to **JSON**, ships the arch **as a string** over **RPC**, and **OWL** (JavaScript) turns that string into a **render function** that calls `document.createElement` to build the **native HTML** the browser finally paints. Files → records → string → JS → DOM.

Odoo String Reference Guide — "where does this string point?"

Superpower: every string in a `.xml` / `.py` / controller belongs to **one of ~9 namespaces**, and each namespace has **one resolver**. Categorize the string → you instantly know where it is **defined** and who **references** it. This is how to read any Odoo file with X-ray vision.

🔑 THE 9 NAMESPACES (the whole game)

#	Namespace	Looks like	DEFINED by	RESOLVED by	Referenced by
1	Model name	<code>qms.queue</code>	<code>_name="qms.queue"</code>	the registry (<code>ir_model</code>)	view <code>model</code> , action <code>res_model</code> , <code>env[...]</code> , <code>Many2one(...)</code> , <code>_inherit</code>
2	Field name	<code>q_name</code>	<code>q_name = fields.Char()</code>	the model's fields (<code>ir_model_fields</code>)	<code><field name="..."></code> , <code>domains</code> , <code>group_by</code> , <code>_rec_name</code> , <code>@api.depends</code> , <code>order</code> , <code>related</code>
3	XML-id	<code>action_qms_queue</code>	<code>id="..."</code> on a record	ir_model_data (<code>xml_id→row id</code>)	<code>ref="..."</code> , <code><menuitem action></code> , <code>search_view_id</code> , <code>parent_id</code> , <code>groups</code> , <code>t-call</code>
4	Method name	<code>_compute_ticket_count</code> , <code>create</code>	<code>def ...()</code> on the class	getattr on the model	<code>compute=</code> , <code>button name=</code> , <code>RPC call_kw</code> (public only)
5	Widget name	<code>handle</code> , <code>statusbar</code>	JS, in the web addon	the JS fields registry	<code>widget="..."</code>
6	Template id	<code>website.layout</code> , <code>qms.qms_book_form</code>	<code><template id=...></code> (qweb view)	XML-id (<code>ir_model_data</code>)	<code>t-call</code> , <code>request.render(...)</code>
7	URL / route	<code>/qms/book</code>	<code>@http.route("/qms/book")</code>	the routing map	<code><form action></code> , <code>links</code> , <code>website.menu.url</code> , <code>redirects</code>
8	Context var	<code>queues</code> , <code>error</code>	the controller's dict / parent scope	the render context (that render only)	QWeb <code>t-foreach</code> , <code>t-if</code> , <code>t-esc</code>
9	View type	<code>list</code> , <code>form</code> , <code>search</code>	JS views registry	<code>view_mode</code> , <code><list>/<form></code> tag	the action / the arch
—	Free text	<code>string="Name"</code> , <code>help</code> , <code><field name="name">qms.queue.form</field></code>	nowhere — it's a label	nobody	(CSS <code>class="..."</code> is the exception → resolved in CSS bundles)

Predict in 2 steps: (a) which of the 9 is it? (b) use that row's "defined / referenced" columns.

THE FOUR "HUB" STRINGS (most connections flow through these)

```
MODEL NAME "qms.queue" ← the biggest hub
  defines: the table, the registry class
  pulled by: view.model, action.res_model, env["qms.queue"], Many2one("qms.queue"),
             One2many(...,...), _inherit, _name everywhere

FIELD NAME "q_name"
  defines: a column + metadata
  pulled by: <field name="q_name"/>, domain [{"q_name","=",..}], _rec_name="q_name",
             @api.depends("q_name"), order="q_name", related="...q_name"

XML-ID "action_qms_queue" (resolved via ir_model_data)
  defines: a named pointer to a row
  pulled by: ref="action_qms_queue", <menuitem action="action_qms_queue">,
             search_view_id ref, parent_id ref, groups="module.group_xxx"

URL "/qms/book" (resolved via the routing map)
  defines: an endpoint
  pulled by: <form action="/qms/book/submit">, website.menu url, <a href>, redirects
```

If a string isn't one of these four, it's usually a **method**, **widget**, **template id**, **context var**, **view type**, or **free text** — check the table.

WALKTHROUGH 1 — qms_queue_views.xml (every string annotated)

```
<record id="view_qms_queue_form"                <!-- ③ XML-id → ir_model_data (referenced by menus/actions) -->
  model="ir.ui.view"                             <!-- ① model → which model this RECORD is created in -->
  <field name="name">qms.queue.form</field>        <!-- FREE TEXT → just a label, references nothing -->
  <field name="model">qms.queue</field>            <!-- ① model name → registry (this view is FOR qms.queue) -->
  <field name="arch" type="xml">
    <form>                                         <!-- ⑨ view type → JS views registry (FormRenderer) -->
      <field name="q_name"/>                       <!-- ② field name → resolved on qms.queue (must exist) -->
      <field name="quid" readonly="1"/>           <!-- ② field + attr override -->
      <field name="service_id"/>                  <!-- ② field (Many2one → its relation points to ①qms.service) -->
    </form>
  </field>
</record>

<record id="action_qms_queue" model="ir.actions.act_window"> <!-- ③ XML-id + ① model -->
  <field name="res_model">qms.queue</field>           <!-- ① model name → what the action opens -->
  <field name="view_mode">list,form</field>           <!-- ⑨ view types -->
  <field name="search_view_id" ref="view_qms_queue_search"/> <!-- ③ XML-id REFERENCE → ir_model_data -->
</record>
```

Search-view extras:

```
<filter name="group_service" context="{ 'group_by': 'service_id' }"/>
  <!-- name=... is a filter id (local) ; 'service_id' inside context = ② field name -->
```

WALKTHROUGH 2 — qms_queue.py (every string annotated)

```
_name = "qms.queue"          # ① DEFINES the model-name hub (everything points here)
_rec_name = "q_name"          # ② field name (which field is the display name)
_order = "quid"               # ② field name (sort)

q_name = fields.Char(string="Name") # field NAME=q_name (②) ; string="Name" = FREE TEXT label
service_id = fields.Many2one("qms.service") # ① references ANOTHER model name
ticket_ids = fields.One2many("qms.ticket", "quid") # ① model + ② inverse field name on qms.ticket
ticket_count = fields.Integer(compute="_compute_ticket_count") # ④ METHOD name (string)

@api.depends("ticket_ids")      # ② field name(s) that trigger recompute
def _compute_ticket_count(self): # ④ method DEFINITION (private "_" → NOT RPC-callable)
    for q in self: q.ticket_count = len(q.ticket_ids)

def create(self, vals_list):     # ④ public method → IS an RPC endpoint (call_kw)
    last = self.search([], order="quid desc", limit=1) # ② field name in order
    ...
```

WALKTHROUGH 3 — controllers/main.py (every string annotated)

```
@http.route("/qms/book",          # ⑦ URL → registered in the routing map
             type="http",         # request type (http = returns HTML)
             auth="public",       # auth mode (public = anonymous allowed)
             website=True,        # website integration
             methods=["GET"])     # HTTP verb(s)
def qms_book_form(self, **kw):
    queues = request.env["qms.queue"].sudo().search([]) # ① model name → registry
    return request.render(
        "qms.qms_book_form",      # ⑥ TEMPLATE id → a qweb ir.ui.view (via ir_model_data)
        { "queues": queues,       # ⑧ CONTEXT VAR "queues" → used by t-foreach="queues"
          "error": kw.get("error")}, # "error" → used by t-if="error" in the template
    )
```

And in the template (`qms_public_templates.xml`):

```
<t t-call="website.layout">
<t t-foreach="queues" t-as="queue">
  <option t-att-value="queue.id">
<form action="/qms/book/submit" ...>
<!-- ⑥ template id REFERENCE → website addon -->
<!-- ⑧ context var "queues" (from the dict above) -->
<!-- queue.id → ② field on the looped record -->
<!-- ⑦ URL → matches another @http.route -->
```

THE PREDICTION ALGORITHM (apply to any string)

See a string in an Odoo file. Ask, in order:

- | | |
|--|---|
| 1. Is it dotted lowercase like "a.b"? | → ① MODEL NAME (registry / ir_model) |
| 2. Is it inside name="..." on <field>, or in a domain/order/depends/group_by? | → ② FIELD NAME (the current model's fields) |
| 3. Is it an id="..." or a ref="..." / action= / parent_id/groups/search_view_id? | → ③ XML-ID (ir_model_data) |
| 4. Is it in compute= / a button name= / called via call_kw? | → ④ METHOD (getattr on the model) |
| 5. Is it in widget="..."? | → ⑤ WIDGET (JS fields registry) |
| 6. Is it in t-call / request.render(...)? | → ⑥ TEMPLATE ID (qweb view, ir_model_data) |
| 7. Does it start with "/"? | → ⑦ URL (routing map) |
| 8. Is it a t-foreach/t-if/t-esc variable? | → ⑧ CONTEXT VAR (the render dict / scope) |
| 9. Is it list/form/kanban/search? | → ⑨ VIEW TYPE (views registry) |
| else (string=, help=, a <field name="name"> label) | → FREE TEXT (a label; references nothing) |

Then: "DEFINED by" + "Referenced by" come straight from the namespace's table row.

DIRECTION CHEAT (refers-TO vs referred-BY)

<code>_name="qms.queue"</code>	— is referred BY →	views, actions, relations, env[...]
<code><field name="model">qms.queue</code>	— refers TO →	the model <code>_name</code>
<code>id="action_qms_queue"</code>	— is referred BY →	menus, ref=, search_view_id
<code>ref="view_qms_queue_search"</code>	— refers TO →	the id="view_qms_queue_search"
<code>compute="_compute_x"</code>	— refers TO →	def <code>_compute_x</code> on the same class
<code>request.render("qms.tpl", {k})</code>	— refers TO →	template id "qms.tpl"; k = vars for it
<code>@http.route("/x")</code>	— is referred BY →	forms/links/menus that point to /x

✓ The takeaway

There is no mystery string in Odoo. Each one is in **exactly one namespace**, resolved by **exactly one mechanism** (registry, `ir_model_data`, the model's fields, the routing map, a JS registry, or the render scope). Memorize the **9 namespaces** + the **4 hubs** (model name, field name, XML-id, URL), and you can trace any identifier in any `.xml` / `.py` / controller to both ends of its link.